Sensor-Based Human Activity Recognition Using a Gaussian Hidden Markov Model

Author: Gaddiel Irakoze

1. Background and Motivation

Human activity recognition (HAR) can infer physical behavior from motion measurements without continuous video surveillance. This individual study investigates a low-cost, smartphone-only activity monitor that could support offline mobility and exercise tracking in bandwidth-constrained settings. The proposed use case is recognizing short bouts of walking, jumping, standing and complete inactivity from a phone already carried by the user; it is a proof of concept, not a deployed health or safety device. The central question is whether interpretable time and frequency-domain features, combined with a Hidden Markov Model (HMM), can reliably infer these activities from noisy accelerometer and gyroscope signals.

2. Data Collection and Quality

I collected 58 independently recorded, labeled motion segments (5–10 seconds each) with a smartphone sensor-logging application: 50 for model development and eight reserved as unseen tests (two per activity). Walking was performed at a consistent pace, Jumping as repeated jumps, Standing with the phone held steady at waist level, and Still with the phone resting on a flat surface. Each ZIP archive contains timestamped triaxial accelerometer and gyroscope CSV data, alongside additional sensor metadata. The model uses gravity-compensated linear acceleration (m/s^2) and angular velocity (rad/s).

Activity	Training files	Testing files	Usable training duration
Walking	13	2	94.48 s
Jumping	12	2	93.96 s
Standing	12	2	93.80 s
Still	13	2	97.16 s
TOTAL	50	8	379.40 s

The dataset satisfies the minimum of 90 seconds of usable training data per activity. The held-out recordings were processed using the same fixed signal-processing procedure, but were not used to fit the scaler or HMM. Independent recordings were never concatenated into artificial continuous sessions.

Quality assurance. An automated audit across all 58 archives found zero missing sensor values, infinite readings, invalid or duplicated timestamps. It also found zero gaps greater than 100 ms, recordings outside the 5–10-second overlap criterion, or duplicated sensor recordings. All original archives were preserved separately from processed CSVs.

Required submission evidence

GitHub repository: https://github.com/Gaddy01/Human-Motion-Recognition_HMM.git

Four data-collection videos: Videos folder link

The video folder contains Walking.mp4, Jumping.mp4, Standing.mp4 and Steady_phone_on_flat_surface.mp4. Each video meets the rubric's visible-face, full-body, phone and five-second requirements.

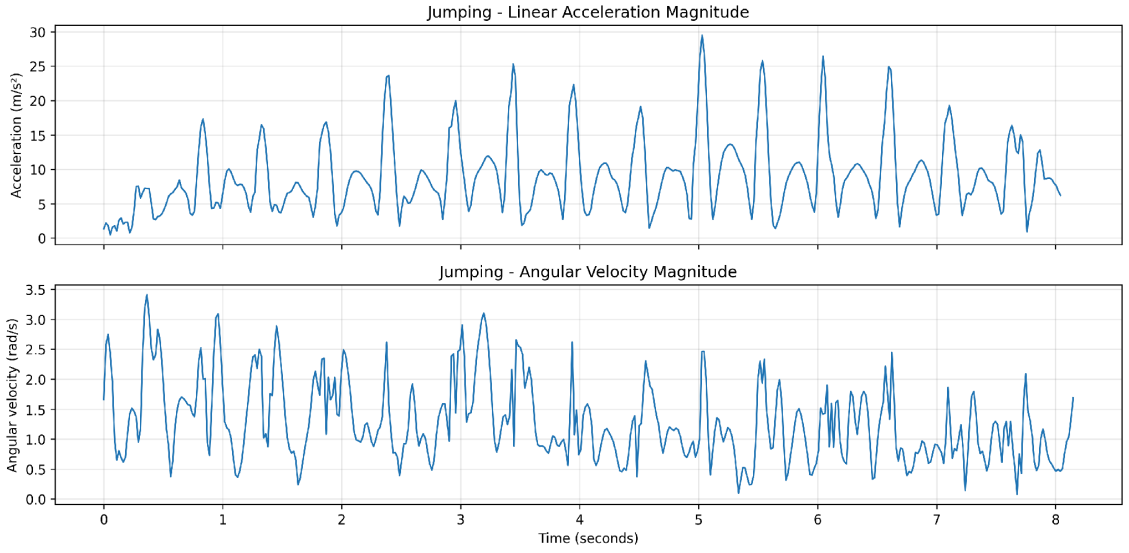


Figure 1. Representative jumping recording: repeated linear-acceleration peaks and variable angular velocity (original sensor data).

3. Signal Pre-processing and Feature Extraction

3.1 Synchronization, filtering and resampling

The original accelerometer frequency varied between approximately 27.55 and 55.10 Hz; gyroscope measurements were approximately 55.10 Hz. For consistent observation timing, both sensors were synchronized on their overlapping timestamp interval and resampled to 25 Hz. A sixth-order zero-phase Butterworth low-pass filter (8 Hz cutoff) preceded linear interpolation to a common 0.04-second grid. Filtering reduces high-frequency contamination and downsampling aliasing while maintaining the lower-frequency motion patterns. The raw archives remain untouched; each processed recording contains six aligned sensor axes and elapsed time.

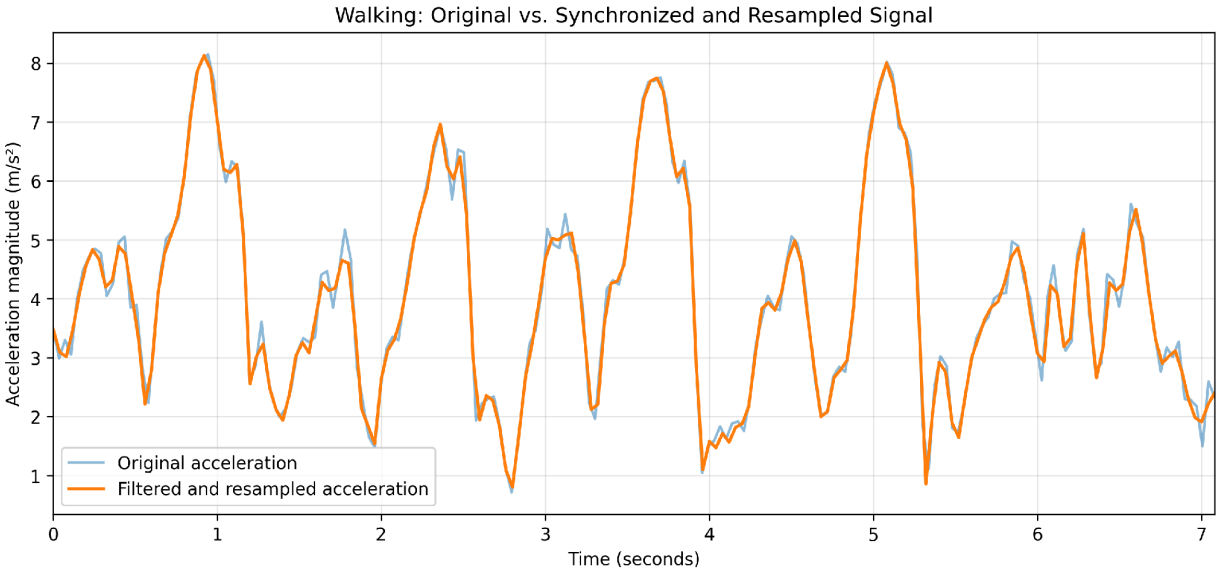


Figure 2. Example walking acceleration magnitude before and after filtering/resampling; major motion patterns are retained (source: preprocessing notebook).

3.2 Window construction

Each recording was segmented independently into 2-second windows (N = 25 Hz × 2 s = 50 samples), with 50% overlap (hop = 25 samples or one second). This provides a new activity description each second while retaining a 0.5 Hz FFT frequency-bin spacing ($\Delta f = 25/50$). Complete windows were used; remainders were not padded. Segmentation generated 303 training windows and 55 test windows, each preserving recording identity, chronological index and activity label.

3.3 Engineered observations and feature selection

Twelve candidate features summarized magnitude, variability, axis relationships and spectral structure. Training-only correlation analysis showed strong redundancy (e.g., acceleration mean and RMS: $r \approx 0.999$). Six complementary candidates were retained for the HMM:

Selected feature	Domain	Reason for inclusion
Acceleration RMS	Time	Overall translational motion intensity
Gyroscope RMS	Time	Overall rotational motion intensity
Acceleration X–Y correlation	Time	Joint movement across axes
Acceleration dominant frequency	FFT	Principal repetitive movement rhythm
Acceleration spectral energy	FFT	Power of acceleration fluctuations (0.5–8 Hz)
Gyroscope spectral energy	FFT	Power of rotational fluctuations (0.5–8 Hz)

The FFT was applied after removing each signal's mean; energy was computed from normalized one-sided power. These features represent one HMM observation vector per window. Z-score normalization was fitted on training windows only, then applied unchanged to test windows, preventing test-distribution leakage.

3.4 Feature interpretation and dataset integrity

RMS magnitude captures overall motion energy without depending on a single phone axis; gyroscope RMS complements it with rotational intensity. X–Y correlation can help separate movement patterns, although it is sensitive to orientation. Dominant frequency detects rhythmic motion, and spectral energy measures the strength of these fluctuations rather than mistaking a weak noise peak for meaningful activity. Thus no single feature is expected to classify all four states on its own. The full 12-feature tables remain archived for reproducibility, while the selected six-feature configuration is recorded separately. Feature rows retain their recording ID and window index so the HMM can reconstruct rather than concatenate independent time series.

4. Hidden Markov Model Setup and Implementation

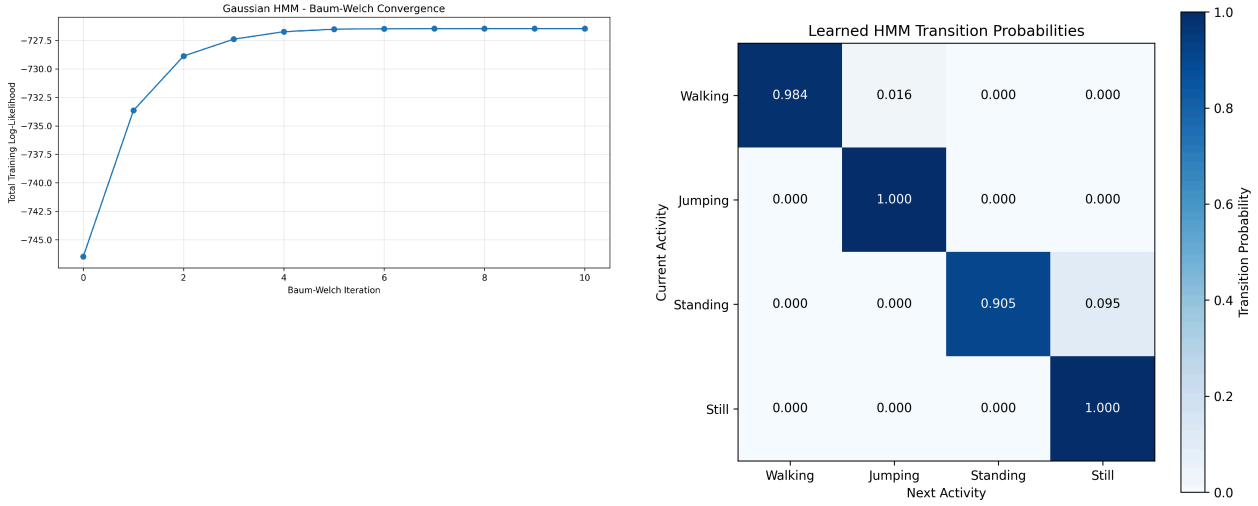
The HMM uses four hidden states Z = {Walking, Jumping, Standing, Still}. Each observation is a six-dimensional, standardized feature vector from one window. The model parameters are the initial-state distribution π, transition matrix A and state-conditional Gaussian emissions B. Each state has its own six-dimensional mean and diagonal covariance matrix; a variance floor of 0.05 prevents degenerate densities. Emission means and variances were initialized from labeled training windows, whereas π began uniformly and A began with 0.98 self-transition probability and equal small off-diagonal probabilities.

Algorithm choice. Baum–Welch (Expectation–Maximization) learns the HMM parameters from the 50 separate training sequences; a log-domain Forward–Backward recursion computes state and transition posteriors. The log-domain Viterbi algorithm then decodes the highest-scoring state path for each recording. Baum–Welch is appropriate for maximum-likelihood parameter estimation; Viterbi is appropriate for sequence-level classification, whose performance is assessed using accuracy, sensitivity and specificity. All HMM routines were implemented from scratch with NumPy (without hmmlearn).

For a state k, the emission density is a six-dimensional normal distribution N(x; μ□, diag(σ²□)). The Forward–Backward expectation step yields posterior state occupancies and expected transitions; the maximization step updates π, A, Gaussian means and variances. Computation in log space avoids numerical underflow when probabilities from successive windows are combined. The fitted parameters remain fixed during final evaluation; the test set is decoded but never used to update the model.

4.1 Training behavior

Training used 303 feature windows from 50 independent recordings. Baum–Welch stopped after 10 iterations: total log-likelihood improved from −746.4748 to −726.4631 (+20.0117), and the final change of 0.00002714 fell below the convergence tolerance of 10^{-4} (maximum 150 iterations). Convergence establishes numerical stability under this initialization, not global optimality or unseen-data performance. Training window accuracy was 92.74%.



Figures 3–4. Baum–Welch log-likelihood converged (left); the fitted transition matrix is dominated by self-transitions (right).

4.2 Learned transition and emission structure

The estimated self-transition probabilities are 0.9843 (Walking), ≈1.0000 (Jumping), 0.9050 (Standing) and ≈1.0000 (Still). Two noticeable off-diagonal terms are Walking→Jumping = 0.0157 and Standing→Still = 0.0950. These are model-internal estimates, not observed behavioral transition frequencies: each labeled recording contains only one sustained activity. Initial-state probabilities were relatively balanced (Walking 0.2798; Jumping 0.2202; Standing 0.2614; Still 0.2386).

Reproducibility: trained parameters, fitted standardization statistics, feature definitions and evaluation scripts are saved in the project artifacts. Recording boundaries were preserved throughout training and decoding.

5. Results and Interpretation on Unseen Data

The final frozen model was evaluated on eight held-out recordings containing 55 overlapping windows. It correctly classified 47/55 windows (85.45%) and seven of eight recordings (87.50%; recording label assigned by majority vote of Viterbi states). Macro F1-score was 0.8444. The 7.29-percentage-point train–test window-accuracy gap suggests moderate generalization loss, but only eight independent test recordings were available.

5.1 Required classwise evaluation

Activity	Test windows	Sensitivity	Specificity	Overall accuracy*
Walking	13	53.85%	95.24%	85.45%
Jumping	15	86.67%	100.00%	96.36%
Standing	13	100.00%	85.71%	89.09%
Still	14	100.00%	100.00%	100.00%

*Activity-level “overall accuracy” is one-versus-rest accuracy (TP + TN divided by all 55 test windows); the global four-class window accuracy is 85.45%. Sensitivity = TP/(TP + FN); specificity = TN/(TN + FP).

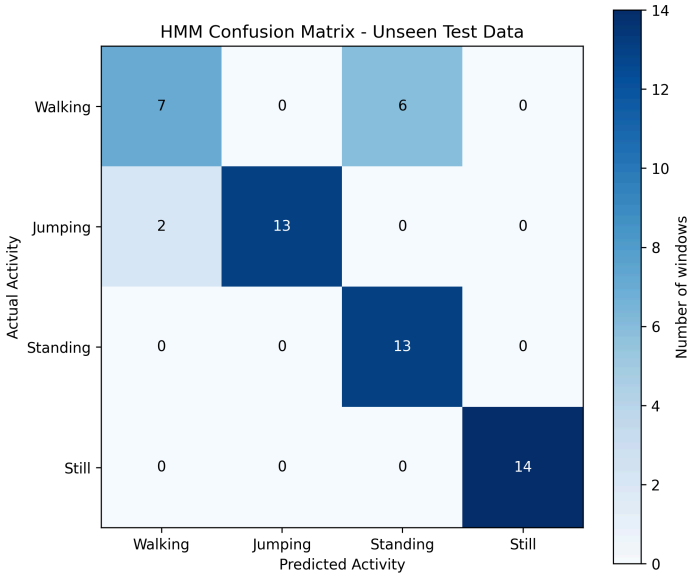


Figure 5. Test confusion matrix (actual classes by row; predicted classes by column).

The matrix reveals 6 Walking→Standing errors and 2 Jumping→Walking errors; all 13 Standing and all 14 Still windows were correct. Walking had the lowest sensitivity (53.85%), whereas Still achieved 100% sensitivity and specificity on this small test set. Standing’s perfect sensitivity coexisted with imperfect specificity (85.71%) because six Walking windows were falsely labeled Standing.

The Viterbi plots corroborate these findings: both Jumping recordings begin with one window predicted Walking, then stabilize as Jumping; Standing and Still remain correctly decoded throughout. Walking_test_01 is correct for all seven windows, but Walking_test_02 is predicted Standing for all six windows. Thus the largest failure is not a single uncertain window, but sustained confusion throughout one complete recording.

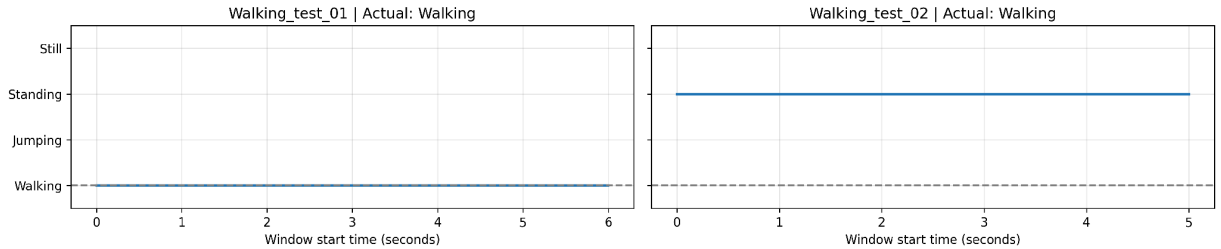


Figure 6. Viterbi paths for the two unseen Walking recordings: Walking_test_01 is correct (left); Walking_test_02 remains incorrect in Standing (right).

6. Discussion, Error Analysis and Conclusion

6.1 Why Walking_test_02 failed

A post-evaluation diagnostic compared the fitted Walking and Standing Gaussian log-densities without changing parameters. For Walking_test_02, the cumulative emission log-score favored Standing by +4.6548. The initial/self-transition component favored Walking by −0.4881, leaving a +4.1666 constant-path advantage for Standing. Acceleration dominant frequency was the strongest Standing-favoring feature (+5.7283 average log-density difference per window), outweighing Walking-favoring acceleration RMS (−2.7758) and gyroscope RMS (−3.4038). By contrast, Walking_test_01 strongly favored Walking (−138.3795 cumulative emission difference). Hence, the misclassification primarily reflects atypical emission evidence, not a stronger transition prior for Standing.

6.2 Emission evidence

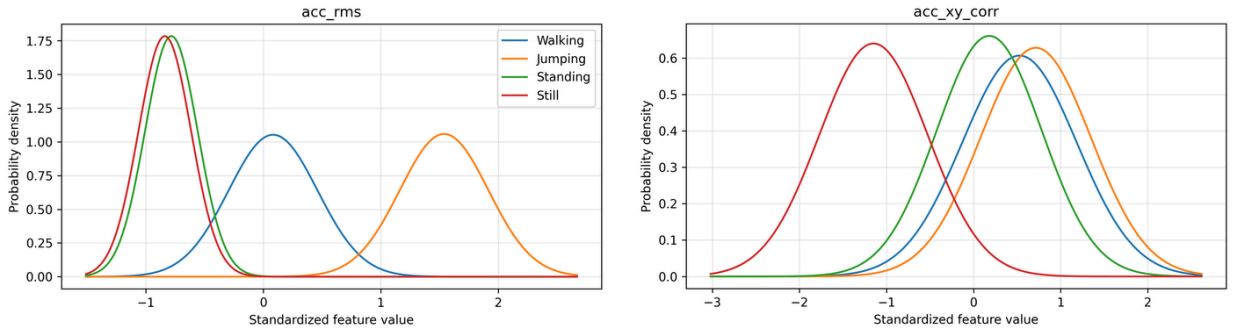


Figure 7. Illustrative fitted Gaussian emission densities: acceleration RMS (left) and X–Y acceleration correlation (right); colors correspond to the four learned states (source: modeling notebook).

Acceleration RMS separates high-intensity Jumping and Walking from low-intensity Standing and Still, but the latter two overlap strongly. X–Y correlation adds different discriminative information, especially for Still; it is nevertheless sensitive to phone orientation. Other spectral features contribute movement rhythm and rotational-power information, although correlated features and the diagonal-covariance assumption remain simplifications.

6.3 Limitations and improvements

The model's strong self-transition structure reflects the collection protocol rather than real inter-activity changes, so continuous-state switching was not meaningfully validated. Additional limitations are small independent-test size (two recordings per class), overlapping and therefore correlated windows, possible differences in phone orientation or walking intensity, frequency peaks dominated by noise in stationary recordings, and Gaussian-distribution assumptions. A stronger follow-up would collect participants and sessions with diverse phone placements, include deliberately mixed-activity sequences, and compare alternative windows/features using recording-level validation confined to training data. The held-out eight recordings should not guide retroactive tuning.

6.4 Conclusion

This individual investigation demonstrates an end-to-end, reproducible HAR pipeline: 58 validated recordings, synchronized six-axis signals at 25 Hz, 2-second overlapping feature windows, six selected engineered observations and a four-state Gaussian HMM trained from scratch. Baum–Welch converged in 10 iterations, and the frozen model achieved 85.45% unseen-window accuracy and 87.50% unseen-recording accuracy. Still was easiest to recognize; Walking was hardest, with one entire walking recording confused for Standing. These results support the feasibility of lightweight smartphone-based activity inference while making clear the need for more diverse data and true activity-transition trials before real-world deployment.

7. AI disclosure

AI tools (chatgpt) assisted with code debugging, interpretation of selected results, and report drafting. However, I fully understand the methodology, algorithms, code, and findings presented in this formative assessment and can independently explain and justify each component. I take full responsibility for the accuracy, originality, and integrity of the submitted work.